Familiarity Routing with Label Copying for Efficient Real-Time DDoS Detection on Hadoop, Spark and Kafka

Nirmal A R*, Dheeraj Kumar G*, Meera S Raj* and Vishnu Vardhan N*
*School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore, India
CB.AI.U4AID24001, CB.AI.U4AID24017, CB.AI.U4AID24031, CB.AI.U4AID24072
Team 17, 23AID302 Big Data Analytics

***Abstract*—A DDoS detector is usually trained once and then asked to judge traffic that keeps changing. We built a complete big-data pipeline on Hadoop HDFS, Apache Spark and Apache Kafka for the CSE-CIC-DDoS2019 dataset (51.2 GB, 70.4 million flow records over two capture days) and used it to measure what happens when five Spark MLlib models trained on the first day are tested on the second. Tested on the same day, the models reach 0.89 to 0.98 macro-F1. On the later day four of them fall to between 0.46 and 0.76; only the random forest holds at 0.967, and it is the slowest model. We found that 96 to 99.9% of every model's day-2 mistakes fall in the 22.6% of traffic that lies far from the day-1 training data, measured by distance to the nearest of 50 K-Means centres. We use this in two ways. A *familiarity router* lets a cheap day-1 model answer familiar traffic and sends unfamiliar traffic to the random forest. *Label copying* answers an unfamiliar connection with the true label of a nearly identical connection that has already been labelled, and asks for new labels through Kafka only where the forest had to answer. Nothing is retrained. Running live over the whole of day 2 through Kafka, the combination lifts the neural network from 0.685 to 0.983, gradient-boosted trees from 0.485 to 0.974 and the random forest itself from 0.967 to 0.982, using 1,000 labels. Measured on 5,964,207 connections in stream order, scoring becomes 1.60 times faster than running the forest on everything, and 2.76 times faster during the Syn flood, because the forest is needed for only 2.11% of connections.**

***Index Terms*—DDoS detection, Apache Spark, Apache Kafka, HDFS, Structured Streaming, MLlib, concept drift, K-Means, nearest neighbour, CIC-DDoS2019**

I. Introduction

Distributed denial-of-service (DDoS) attacks flood a target with traffic from many machines until it can no longer serve real users [1]. Detecting them from flow records is a well-studied machine-learning task, and on public datasets such as CIC-DDoS2019 [2] published models often report accuracy above 99%. Most of these results come from a random split of one dataset, so the test traffic was captured at the same time and on the same network as the training traffic. Sommer and Paxson pointed out long ago that intrusion detection is a hard setting for machine learning because the traffic a detector meets in operation differs from what it was trained on [3], and Arp et al. list mixing past and future data as one of the most common pitfalls in security ML [4].

CIC-DDoS2019 gives a direct way to test this, since it was captured on two separate days with partly different attacks. When we trained five standard Spark MLlib models on day 1 and tested them on day 2, four of the five lost most of their accuracy. This report describes the system we built to study that drop at full data size, what we found when we looked at where the models fail, and two simple mechanisms that use that finding to make live detection both more accurate and cheaper.

A. Why big data tools are needed

The raw dataset is 51.2 GB in 18 files, two formats and 70.4 million rows, which does not fit in the memory of one process on our 15 GB laptop. Cleaning it requires grouping all copies of each connection together, and training needs many passes over millions of rows. The live part has to score thousands of connections per second as they arrive. These are the three classic properties: volume (51.2 GB), variety (CSV for day 1, nested JSON with missing values for day 2) and velocity (a stream replayed through Kafka in time order). We therefore stored the data in HDFS [5], processed it with Spark DataFrames [6], [7], trained with MLlib [8], and streamed it with Kafka [9] and Spark Structured Streaming [10].

B. Objectives

1) Build an end-to-end pipeline that ingests, cleans and stores the full dataset on HDFS and trains five MLlib models with Spark.
2) Test the models honestly on a later capture day, not only on a random split of the same day.
3) Stream day 2 through Kafka and score it live, writing results to PostgreSQL and Grafana.
4) Keep detection accurate on unfamiliar traffic without retraining, and reduce the work done per connection.
5) Explain each decision to the person whose network it is, through a small website with a local language model.

C. Contributions

- A measurement, on all 19.9 million day-2 flows, showing that almost all mistakes of every model are concentrated in traffic that is far from the day-1 data (Section VII).
- A familiarity router that uses this distance to decide which model answers each connection (Section VIII).

- Label copying, which reuses a small number of true labels for later connections that look the same (Section IX).
- A live implementation in Scala on Spark and Kafka, with timings for every batch job and for each streaming tick (Sections V and X).

II. Related Work

DDoS datasets and detection. CIC-DDoS2019 was introduced by Sharafaldin et al. together with a taxonomy of reflection and exploitation attacks [2]. Its flow features come from CICFlowMeter [11]. Ring et al. survey the network intrusion datasets in common use and note that most capture a short period in one testbed [12]. Awan et al. built real-time DDoS detection on Spark ML with random forest and a neural network and report 99.5% accuracy [13]; their evaluation uses one dataset with no test on traffic from a later period, which is the gap our cross-day experiment looks at.

Big data systems. HDFS stores large files as replicated blocks across data nodes [5]. Spark keeps intermediate data in memory as resilient distributed datasets [6], offers a DataFrame API with a query optimiser [7] and a unified engine for batch, streaming and ML [14]. Structured Streaming treats a stream as a table that grows in micro-batches [10]. Kafka is a partitioned, replicated log built for high-volume event data [9]. MLlib provides the distributed learners we use [8].

Models. Random forests [15] and gradient boosting [16] are the two tree ensembles we compare. The familiarity check uses K-Means [17] with the scalable k-means‖ initialisation that Spark implements [18]. Label copying is a one-nearest-neighbour rule [19]. Sending easy inputs to a cheap model and hard ones to a strong model is the same idea as a detection cascade [20].

Drift. Gama et al. survey methods for data whose distribution changes over time [21]. In security, TESSERACT shows that classifiers decay over time and that random splits hide this; its remedies are retraining and active learning [22], [23]. Transcend uses conformal evaluation to reject predictions that cannot be trusted [24], and CADE trains a contrastive network to measure how far a sample is from the training data and flags drifting samples for an analyst [25]. A common baseline for spotting likely mistakes is the model's own confidence [26]. Our gate is closest to CADE in spirit, but uses plain K-Means distance, needs no extra network, and does not stop at flagging: the flagged traffic is answered in the same tick.

III. Dataset

CSE-CIC-DDoS2019 [2] contains flow records produced by CICFlowMeter from two days of testbed traffic. Each record has identifiers (flow ID, timestamp, IP addresses, ports, protocol), 80 numeric flow measurements and a label.

- **Day 1, 12 January 2019**, the training day: 11 CSV files, 50,063,112 flows, benign traffic and 12 attack families (DNS, LDAP, MSSQL, NetBIOS, NTP, SNMP, SSDP, UDP, Syn, TFTP, UDPLag, WebDDoS).

TABLE I
Day-2 flows after cleaning (19,881,726 in total)

Type	Flows	Type	Flows
MSSQL	5,787,453	LDAP	1,915,122
Syn	4,413,105	Portmap	186,960
UDP	3,867,155	Benign	52,561
NetBIOS	3,657,497	UDPLag	1,873

- **Day 2, 11 March 2019**, the testing day: 7 files, 20,364,525 records, benign traffic and Portmap, NetBIOS, LDAP, MSSQL, UDP, UDPLag and Syn. Portmap never appears on day 1.

To exercise semi-structured input, we stored day 2 as JSON lines with nested groups (`flow.*`, `packet_length.*`, ...) and part of its values removed, so the pipeline had to read two formats and recover missing values. In total the raw data is 51.2 GB with 70,427,637 rows. Benign traffic is under 1% of flows. Table I lists day 2 after cleaning.

Key attributes. We keep 55 numeric features as model inputs: protocol, flow duration, forward and backward packet counts and byte totals, packet-length statistics (min, max, mean, standard deviation, variance), bytes and packets per second, inter-arrival-time statistics, TCP flag counts (SYN, ACK, URG, CWE, forward PSH), header lengths, down/up ratio, average packet size, active data packets, and active and idle time statistics. We removed IP addresses, ports, flow IDs, timestamps and TCP window fields, because they describe the testbed rather than the behaviour of the traffic and a model could memorise them. Columns that were constant, or exact copies of another column (for example the four subflow counts, which equal the totals), were also dropped.

Two properties of the data shaped the design. First, 84% of day-1 flows have exactly the same 55 measurements as another flow; a flood repeats itself. Second, 69% of day-1 flows share their measurement vector with a flow of a *different* attack family, while vectors labelled both benign and attack are very rare. Attack versus benign is therefore well determined by the features, but the attack family is not, which is why binary detection is our main task.

IV. System Architecture

Everything runs on one laptop (15 GB RAM, 11 GB given to WSL2) with Docker Compose. There are 12 containers: an HDFS name node and three data nodes (Hadoop 3.2.1), a Spark 3.5.5 standalone master, one worker and a history server, a Kafka 3.7.0 broker in KRaft mode, PostgreSQL 16, Grafana 11.2, PostgREST for read-only REST access to the result tables, and nginx for the website. The Scala 2.12 code is built with sbt-assembly into one jar that every job uses. Fig. 1 shows how data moves through the system.

Table II gives the resources given to Spark. With two executors of three cores, at most six tasks run at the same time. That number explains most of the partition choices below.

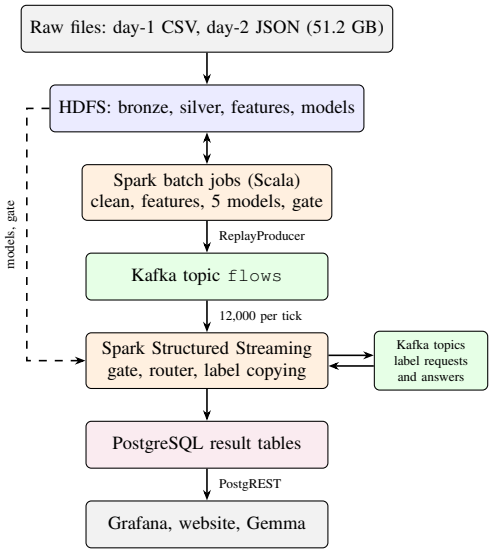


Fig. 1. System architecture. Batch jobs read and write HDFS. The stream reads Kafka, uses the models and gate saved in HDFS, exchanges label requests and answers through two more Kafka topics, and writes results to PostgreSQL.

TABLE II
CLUSTER SETTINGS

Setting	Value
Spark worker	6 cores, 5 GB
Executors	2, each 3 cores and 2,500 MB
Driver	2 GB, client mode
Shuffle partitions	24 default, 96 heavy jobs, 6 live
HDFS	3 data nodes, 128 MB blocks, replication 1
HDFS used	79.66 GB (25.9 GB per data node)
Kafka topics	`flows`, `label_requests`, `labels`; 1 partition each
Event logs	`hdfs:///spark-logs`, read by the history server

Replication is set to 1 because all three data nodes share one physical disk, so extra copies would cost space without adding safety.

V. Data Pipeline in Spark

The batch side is a sequence of Scala jobs, each started with `spark-submit`. All of them use the DataFrame API, so Spark's optimiser plans the work. Every intermediate table is written to HDFS as Parquet, a compressed columnar format, which lets later jobs read only the columns they need.

A. Bronze: one schema for two formats

A single list of 86 fields holds, for every column, its clean name, its CSV header and its JSON path. `BuildBronze` reads the day-1 CSV with that mapping and the day-2 JSON with an explicit schema, so Spark does not need an extra pass to infer types. The two days are combined with `unionByName` and written partitioned by capture day:

```
day1.unionByName(day2)
  .write.mode("overwrite").partitionBy("capture_day")
  .parquet("/data/bronze")
```

This job has no shuffle: reading, renaming, casting and `union` are all narrow, and `partitionBy` only decides which folder each task writes into. The largest stage runs 413 tasks, one per 128 MB split of the 51.2 GB input, and writes 413 files. The output is 3.3 GB, 15.5 times smaller than the raw text.

B. Silver: cleaning

`BuildSilver` unifies the label spellings into 14 families and adds a 0/1 attack column. TCP flag counts are set to 0 for non-TCP flows, where they are 0 by definition. Day 2 contains repeated records of the same connection; grouping by the connection key (day, flow ID, timestamp, IPs, ports, protocol) merges 482,799 copies and keeps any value present in one of them. This `groupBy` is the job's main shuffle, 7.31 GB, and it is why the job uses 96 shuffle partitions.

Missing values are then rebuilt from other columns with 32 exact formulas (for example packets per second = packets / duration, variance = std^2), applied in four passes because one recovered value can unlock another. Division by a zero duration produces NaN and infinity: NaN becomes 0 and infinity becomes the largest finite value seen on day 1, so the test day never influences a cleaning choice. Whatever is still missing is filled with the median of that column for the same day and protocol, computed with `percentile_approx`. The silver table has 69,944,838 rows (4.3 GB) and no null features.

C. Features: collapse, split and encode

`BuildFeatures` groups by day, label and all 55 features and counts how often each pattern occurs. This collapses 70 million flows into 11,857,683 distinct rows (8.2 million on day 1, 3.6 million on day 2), and shuffles 9.09 GB. Each row is then put in the train or test split by a hash of its feature values:

```
when(pmod(xxhash64(("capture_day" +: features).map(col): _
    *),
  lit(100)) < 80, "train").otherwise("test")
```

The hash makes the split repeatable, and because duplicates were merged first, the same pattern can never be in both train and test. Without this step most test rows would be exact copies of training rows. A `StringIndexer` and `VectorAssembler` are fitted on the training split and saved, so the live job encodes traffic exactly as training did. The output is partitioned by day and split, which gives $96 \times 2 \times 2 = 384$ files and lets training read only the folders it needs.

D. Partitions, shuffles and caching

Table III lists what the Spark history server recorded for each job. We lowered `spark.sql.shuffle.partitions` from Spark's default of 200 to 24, about four tasks per core, because 200 partitions on six cores would mean many tiny tasks whose fixed cost outweighs their work. The two jobs that shuffle 7 to 9 GB use 96 partitions instead, so each task holds under 100 MB and fits in its share of executor memory; at 24,

TABLE III
BATCH JOBS AS RECORDED BY THE SPARK HISTORY SERVER

Job	Time	Jobs	Stages	Tasks	Shuffle
BuildBronze	8.8 min	7	7	499	none
BuildSilver	45.1 min	32	33	1,038	7.31 GB
BuildFeatures	11.2 min	33	33	1,194	9.09 GB
TrainModelsWhole	102.7 min	$\geq$921	$\geq$946	$\geq$8,486	small
BuildGate	0.6 min	45	67	396	small
ReplayProducer	2.5 min	3	–	18	1 partition

TABLE IV
CACHING CHOICES

Where	Level	Why
BuildFeatures	`DISK_ONLY`	read twice (fit, write); too big for executor memory, cheaper than redoing the 9 GB shuffle
Training data	`MEMORY_AND_DISK`	read hundreds of times, once per iteration or tree level
Gate sample	default persist	K-Means reads it once per iteration
Each live tick	`local-Checkpoint`	about 16 actions per tick; cuts the plan so Kafka and the models run once
Label memory	broadcast	one read-only copy per executor

BuildSilver's tasks would hold around 300 MB each and spill even more than the 6.96 GB they did. When reading back many small files, Spark packs them into splits of up to 128 MB and counts each file as at least 4 MB, which is why the 413 bronze files are read as 41 partitions.

BuildSilver and BuildFeatures show 19 and 34 *skipped* stages. A stage is skipped when a later job needs shuffle output or cached data that an earlier job already produced, so Spark reuses it instead of recomputing it. The job counts are higher than the number of actions in the code because adaptive query execution runs each shuffle stage as its own job, so that it can re-plan the next step from real data sizes.

Caching was chosen per use (Table IV). The rule we followed was simple: cache only what is used more than once, and choose memory or disk by how many times it is read and how large it is.

VI. MODELS AND TRAINING

A. Algorithms and tuning

We train five MLlib classifiers for attack versus benign: logistic regression (100 iterations), a decision tree (depth 12), a random forest, gradient-boosted trees, and a multilayer perceptron with layers 55–64–32–2 on standardised inputs. The forest and boosted trees are tuned with a `TrainValidationSplit` (75/25) over depth $\{8, 12, 16\}\times$ trees $\{50, 100\}$ and depth $\{4, 6\}\times$ rounds $\{20, 40\}$; the chosen settings were depth 16 with 100 trees, and depth 6 with 40 rounds. Settings are chosen by the F1 score of the benign class, because with under 1% benign traffic a model that calls everything an attack would still look accurate.

Class weights make both classes count equally: each row in class c gets weight $N/(2N_c)$. Tuning runs on a sample with

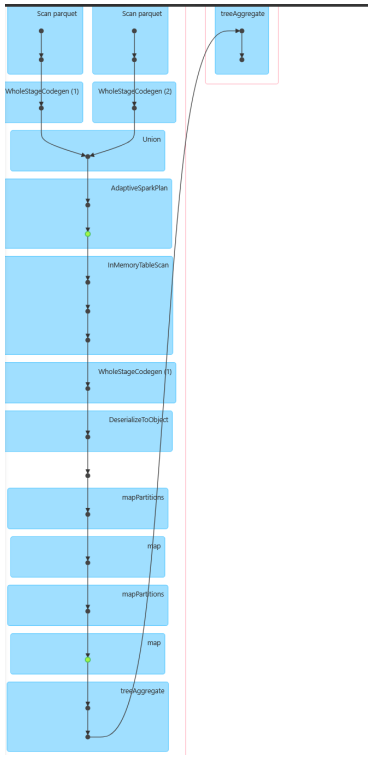


Fig. 2. Spark UI DAG of one training job. Benign and attack rows are scanned and unioned, read back from the cache (green dot), converted to MLlib records, and summed with `treeAggregate` over two stages.

all benign rows and at most 20 attack rows per benign row; `TrainModelsWhole` then retrains every model once on all rows with the chosen settings. A 14-class random forest for the attack family is trained as well.

B. What the Spark DAG of training looks like

MLlib learners are iterative, and every iteration is a Spark job. Logistic regression and the perceptron run one `treeAggregate` per iteration: each of the 11 cached partitions computes a partial gradient, only those partial sums cross the network, and they are combined in a second stage (Fig. 2). Trees grow one level at a time; for each level, every partition counts class statistics per node, feature and bin (up to 32 bins), a `reduceByKey` combines them by node, and the driver picks the best splits. The rows never move, only small summaries do, which is how training on millions of rows finishes on six cores. This is also why TrainModelsWhole ran more than 900 jobs.

C. Same-day and cross-day results

Table V compares the standard test (80/20 split of the same days) with the cross-day test (train on day 1, test on day 2), scored once per distinct pattern. The last column scores every day-2 flow, which is what the live stream sees, because a flood repeats the same pattern millions of times.

On the same days every model looks strong. On day 2 four of them fail badly; boosted trees miss 2.5 million of the attacks. Only the random forest holds up, and it is the

TABLE V
MACRO-F1: SAME-DAY TEST AGAINST CROSS-DAY TEST

Model	Same day	Day 2 (patterns)	Day 2 (flows)
Logistic regression	0.888	0.560	0.458
Decision tree	0.951	0.735	0.756
Random forest	0.981	0.973	0.967
Gradient-boosted trees	0.946	0.716	0.485
Neural network	0.977	0.810	0.685

TABLE VI
DAY-2 ERROR RATE ON FAMILIAR AND UNFAMILIAR FLOWS

Model	Wrong on familiar	Wrong on unfamiliar	Mistakes in unfamiliar
Random forest	0.00%	0.15%	96.3%
Decision tree	0.02%	1.98%	96.2%
Neural network	0.00%	3.38%	99.9%
Gradient-boosted	0.01%	56.37%	99.9%
Logistic regression	0.38%	86.00%	98.5%

most expensive model to run, with 100 trees of depth 16 for every connection. The family classifier reaches 0.648 macro-F1, which matches the label overlap described in the dataset section. Fig. 3 shows the Grafana page with these results, the tuning trials and the forest's feature importances.

VII. WHERE THE MODELS FAIL: THE FAMILIARITY GATE

If a model fails on new traffic, the obvious question is which traffic. We measured how far each day-2 connection is from the day-1 training data. Every feature x is first compressed with $\operatorname{sign}(x)\log(1+|x|)$, because the features range from 0 to millions and the largest would otherwise dominate any distance, and then standardised. K-Means with $k = 50$ is fitted on one million day-1 training patterns. For a connection z, its distance is

$$d(z) = \min_{j=1..50} \|z - \mu_j\|_2,$$

and it is called *unfamiliar* when $d(z)$ exceeds the 99th percentile of the distances of day-1 *test* patterns, which came to $\tau = 5.60$. Test rather than training patterns set the limit because the centres were fitted on the training rows, which therefore sit closer to them than any new data would. The limit is computed by `BuildGate`, saved to HDFS and loaded by the live job; it is not typed into the code.

On all of day 2, 22.6% of flows are unfamiliar. Almost all of it is the Syn flood (99.9% of Syn flows are unfamiliar) and changed benign traffic (70.3%); every other attack is under 9%. Table VI shows the result that the rest of the project builds on: whatever the model, nearly all of its mistakes are in that 22.6%.

The models' own confidence is a much weaker signal here. Flows on which the model's top probability is below 0.9 contain only 3.8% of the decision tree's mistakes and 29.4% of the neural network's: these models are confidently wrong on the new traffic. Distance from the training data finds their mistakes; confidence does not (Fig. 5).

VIII. IDEA 1: THE FAMILIARITY ROUTER

The router follows directly from Table VI. A cheap day-1 model, such as the decision tree, answers the familiar traffic, where it is almost never wrong. Unfamiliar traffic goes to the random forest, the one model that holds up on day 2. Nothing is retrained; the router only decides who answers.

This also answers the obvious question of why the forest is not simply used for everything. On familiar traffic the decision tree is as accurate as the forest (0.02% against 0.00% wrong), but it makes one walk down one tree instead of 100. With the tree in front, the forest is needed for 22.5% of connections instead of all of them.

IX. IDEA 2: LABEL COPYING

The traffic that still reaches the forest is very repetitive: a flood sends the same pattern again and again. If we already know the true label of a connection, a later connection that is almost identical can simply take that label. For each unfamiliar connection we find the nearest labelled example *in the same K-Means cluster* [19], using the same scaled space as the gate. If that example is within the same limit $\tau = 5.60$, its label is copied; otherwise the forest answers.

New labels are asked for only where the forest had to answer. Each tick the live job picks at most 24 patterns, never more than a quarter of that tick's candidates, 1,000 in total, and only when the forest handled more than 1% of the tick. Patterns are taken in turn from each cluster so that the labels cover many kinds of traffic, not just the largest flood. Requests are written to the Kafka topic `label_requests`; a second small stream forwards them to `labels` every two seconds. In our setup the true label from the dataset travels with the request and stands in for a human analyst. The labels are read back on the next tick and broadcast to the executors.

Copying improves the score as well as the speed because the unfamiliar traffic is exactly where models make mistakes, and a true label from a near-identical connection is better than any model's guess there.

X. LIVE STREAMING IMPLEMENTATION

`ReplayProducer` takes a 30% sample of day 2 from silver, moves it into a single partition with `repartition(1)`, sorts it by timestamp and writes each row as JSON to the `flows` topic, 5,964,207 messages. One partition is needed because Kafka only keeps order within a partition, and replaying in time order means each attack arrives when it happened.

`LiveScorer` reads the topic with Structured Streaming:

```
spark.readStream.format("kafka")
  .option("subscribe", "flows")
  .option("maxOffsetsPerTrigger", "12000")
  .option("minPartitions", "6")
```

Each micro-batch, or tick, holds 12,000 connections. The topic has a single Kafka partition, which would normally give one task; `minPartitions` splits each tick's offset range into six, one per core. Each connection is encoded, measured by the gate and scored. The routed answer for an unfamiliar connection is one line:



experiment	training	model	macro_f1	benign_auc_pr
cross_day	all_rows	random_forest	0.973	0.977
cross_day	all_rows	neural_network	0.810	0.844
cross_day	all_rows	decision_tree	0.735	0.189
cross_day	all_rows	gradient_boosted_tre	0.716	0.845
cross_day	all_rows	logistic_regression	0.560	0.374
cross_day	sample	neural_network	0.815	0.915
cross_day	sample	gradient_boosted_tre	0.720	0.723
cross_day	sample	random_forest	0.690	0.959

experiment	model	settings	validation_benign_f	chosen
cross_day	gradient_boosted_tre	maxDepth=6, maxIte	0.981	chosen
cross_day	gradient_boosted_tre	maxDepth=6, maxIte	0.977	
cross_day	gradient_boosted_tre	maxDepth=4, maxIte	0.949	
cross_day	gradient_boosted_tre	maxDepth=4, maxIte	0.941	
cross_day	random_forest	maxDepth=16, numTr	0.997	chosen
cross_day	random_forest	maxDepth=16, numTr	0.996	
cross_day	random_forest	maxDepth=12, numTr	0.994	
cross_day	random_forest	maxDepth=12, numTr	0.993	

family	precision_score	recall_score	f1_score	test_rows
TFTP	0.999	0.998	0.998	877491
UDP	0.765	0.991	0.863	593032
NTP	0.994	0.996	0.995	222686
MSSQL	0.920	0.966	0.942	204087
Syn	0.985	0.997	0.991	179867
SSDP	0.747	0.00100	0.00200	176363
SNMP	0.563	0.884	0.688	22677
DNS	0.774	0.315	0.448	22231

Fig. 3. Grafana “Model Lab”: standard and cross-day results for sample and full training, all detection results, the random forest’s most important features, every tuning trial, and per-family scores.

```
val close = col("nn_distance") <= gate.limit
when(close, col("nn_label"))
  .otherwise(col("random_forest"))
```

All per-tick work happens in `foreachBatch`. The first thing it does is `localCheckpoint()`, which computes the tick once and cuts its lineage; without it every one of the tick’s roughly 16 actions would re-read Kafka and rerun all the models, and a tick took about 8 s instead of 2.4 s. Counts that need many confusion cells are computed in one aggregation instead of one job each, and small lookup tables are collected to the driver and used in a UDF instead of a join, which avoids a shuffle. Results go to six PostgreSQL tables over JDBC. A checkpoint on HDFS stores the Kafka offsets of every tick, so a restarted job continues where it stopped and no connection is skipped; because the PostgreSQL writes happen inside `foreachBatch`, delivery is at-least-once. A typical tick runs 16 to 17 Spark jobs and about 115 tasks, with stages of six tasks or one task where adaptive execution merged small shuffle outputs, and no failed tasks.

The live job deliberately scores every connection with all five models, so the dashboards can show each model alone and with the router side by side. The efficiency figures in Section XI come from a separate job, `EfficiencyCheck`, that runs only the model each connection is routed to.

XI. Results

A. Accuracy on the stream

Table VII gives every model’s macro-F1 on the 5,964,207-connection stream in time order, alone, with the router, and with the router plus label copying. Labels are only usable after they have been asked for.

The router turns three weak models into ones as good as the forest, and copying then lifts every model, the forest included, above what the forest manages alone. Missed attacks with boosted trees in front fall from 758,456 to 850, and with the neural network from 43,219 to 596. False alarms also fall: with the decision tree in front, from 917 to 561. Logistic regression improves but stays lowest, because it is wrong on 0.38% of familiar traffic, where the router still trusts it.

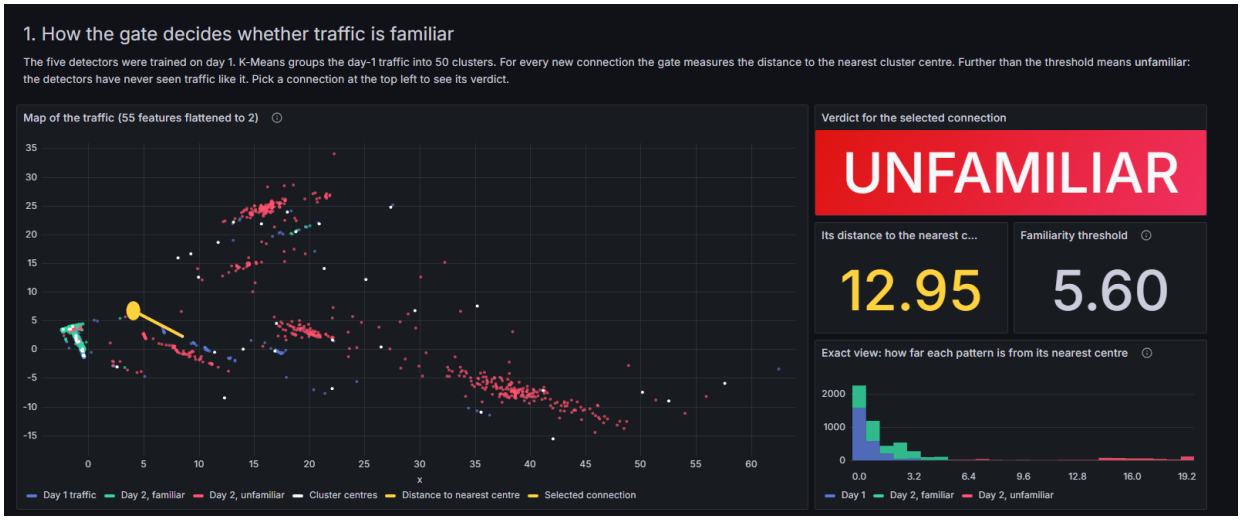


Fig. 4. The gate on our dashboard: a 2-D projection of the 55 features with day-1 traffic, the cluster centres and day-2 traffic coloured by verdict. The selected Syn flood connection is 12.95 from its nearest centre, beyond the 5.60 limit.

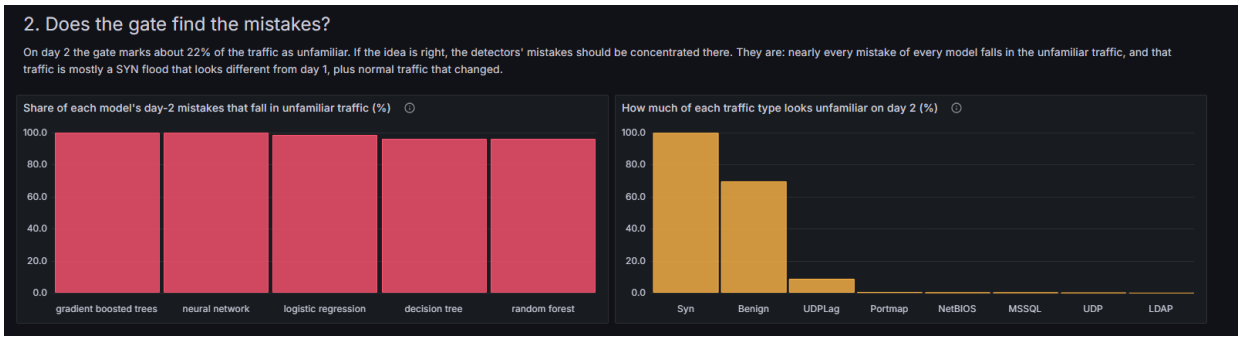


Fig. 5. Left: share of each model's day-2 mistakes that fall in unfamiliar traffic. Right: share of each traffic type that the gate calls unfamiliar.

TABLE VII
MACRO-F1 ON THE STREAM, BY FRONT MODEL

Front model	Alone	Router	Router + copying
Logistic regression	0.458	0.798	0.808
Decision tree	0.756	0.952	0.966
Random forest	0.967	0.967	0.981
Gradient-boosted trees	0.485	0.959	0.973
Neural network	0.685	0.967	0.982

We then ran the full system live over the whole of day 2 (19,881,726 connections, 1,657 ticks, 2.81 s per tick). It used 1,000 labels, sent 1.48% of connections to the forest and answered 21.02% by copying. The neural network went from 0.685 to 0.983, the random forest from 0.967 to 0.982, gradient-boosted trees from 0.485 to 0.974, the decision tree from 0.757 to 0.967 and logistic regression from 0.458 to 0.809 (Fig. 6).

B. Speed

Table VIII times only the model work over the same stream in chunks of 120,000 connections on six cores, with the decision tree in front.

TABLE VIII
EFFICIENCY ON 5,964,207 CONNECTIONS IN STREAM ORDER

Method	Time	Forest	Copied	Speed	Macro-F1
Forest on everything	36.5 s	100%	–	1.00×	0.967
Router	25.8 s	22.50%	–	1.41×	0.952
Router + copying	22.8 s	2.11%	20.38%	1.60×	0.966

Over the whole stream the combination is 1.60 times faster (163,516 against 261,828 connections per second) at the same macro-F1 as the forest, and needs the forest for 2.11% of connections. The 1,000 labels are 0.0168% of the stream; choosing them took 4.5 s in total, in 5 of the 50 chunks. The gain is largest where it matters most. On the 954,274 connections of the Syn flood, with the label memory already filled, scoring took 3.06 s with the forest alone, 2.48 s with the router and 1.11 s with copying, 2.76 times faster.

C. Comparison with retraining

The usual remedy for drift is to retrain on new labels [22]. We built that too: a "specialist" of the same model type, retrained live on all labels so far, answering unfamiliar traffic.



Fig. 6. Grafana results page. Top: the whole of day 2 live, each model alone and with the router, with attacks missed and false alarms. Middle: the efficiency test. Bottom: each idea's contribution to macro-F1.

TABLE IX
RETRAINED SPECIALIST AGAINST ROUTER + COPYING (MACRO-F1)

Model	Alone	Specialist	Router + copying
Random forest	0.967	0.681	0.983
Neural network	0.685	0.509	0.983
Gradient-boosted	0.485	0.659	0.975
Decision tree	0.756	0.900	0.968
Logistic regression	0.458	0.473	0.809

On the same stream it was worse than router plus copying for every model, and for the forest and the neural network it was worse than using no specialist at all (Table IX). Most of its labels were collected before the Syn flood began, so the version in use when the flood arrived called it benign until a newer one was trained. It needed 2,500 labels, about 20 s per retrain and 24 versions, and took about nine ticks to react. The router and copying need no training at all, and on a held-back flood they called 0.06% of it benign, against 16 to 44% for stale specialists.

D. Live dashboards and the website

Fig. 7 shows the router during a live run that starts just before the Syn flood. With boosted trees in front, the model alone drops to about 0.3 macro-F1 as the flood arrives, while the routed system stays close to 1; the panel at the bottom right shows unfamiliar connections being answered mostly from the label memory.

For the person whose network it is, we built a small website, "My Network" (Fig. 8). It replays the last run from PostgreSQL through PostgREST, shows whether the network is protected, how each connection was answered (day-1 model, copied label or forest), and each attack with the features that pushed the deciding model most. Those reasons are computed in the live job by walking each sampled connection down the decision tree or the forest's trees and crediting each feature with the change in attack share at its split. A local Gemma model through Ollama answers questions from the page's facts (Fig. 9), so no traffic leaves the machine.

E. Business insight

For an operator, the result means a cheap model can stay in front for most traffic, the expensive model is paid for only on the small part of traffic that is new, and a few hundred confirmed examples from an analyst are enough to keep a whole attack correctly labelled. The gate also tells the operator, at every tick, how much of the current traffic is unlike anything the detector was trained on, which is useful as an alarm in its own right: in our data that share jumped from almost nothing to two thirds of all traffic when the Syn flood began.



Fig. 7. The router live in Grafana during the Syn flood, gradient-boosted trees in front: labels asked, share of unfamiliar traffic, share sent to the forest, the score over the last 15 ticks, attacks missed per tick, and where unfamiliar connections went each tick.

XII. Challenges and Limitations

Challenges. BuildSilver's recovery step is very wide, and Spark's code generator fell back to a slower mode for it, which together with the 7.31 GB shuffle and 6.96 GB of spill made it the slowest cleaning job. With 11 GB for everything, memory had to be split carefully between Spark, HDFS, Kafka, PostgreSQL and Grafana. A first version of the efficiency test timed label selection together with scoring, which hid the speed-up; we separated the two. After Docker Desktop restarts, the Spark containers sometimes started with empty mounted folders and had to be restarted.

Limitations. We used one dataset, captured in a testbed over two days. The labels that copying uses come from the dataset and stand in for an analyst. The whole system runs on one machine, so Kafka has one partition and HDFS one replica. The live job runs all five models for comparison, so the speed-up is measured by a separate job rather than inside the live dashboards. Attack-family attribution is limited by the dataset itself, since the same packet pattern carries different family labels. The system detects attacks; it does not block them.

XIII. Conclusion and Future Work

Tested on a later day, four of five standard DDoS detectors lost most of their accuracy, and nearly all of their mistakes were in traffic far from what they were trained on. Measuring that distance with plain K-Means let us route each connection to the model that can answer it, and copying a few true labels to near-identical connections fixed most of what remained. Running live through Kafka and Spark over all of day 2, every model improved, the forest included, without any retraining, and scoring became 1.60 times faster over the day and 2.76 times faster during the flood.

Future work: replace the dataset's labels with a real analyst loop; run on a multi-node cluster with a partitioned Kafka topic; update the familiarity limit online as traffic changes; test on other datasets such as CIC-IDS2017; and connect the detector to mitigation, so flagged traffic can be rate-limited or blocked.

Acknowledgment

We thank our course faculty for 23AID302 Big Data Analytics for the feedback that pushed us to test the models on a different day and to think about the person whose network is being protected.

References

[1] J. Mirkovic and P. Reiher, "A taxonomy of DDoS attack and DDoS defense mechanisms," *ACM SIGCOMM Computer Communication Review*, vol. 34, no. 2, pp. 39–53, 2004.

[2] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. IEEE 53rd Int. Carnahan Conf. Security Technology (ICCST)*, Chennai, India, 2019, pp. 1–8.

[3] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security and Privacy*, 2010, pp. 305–316.

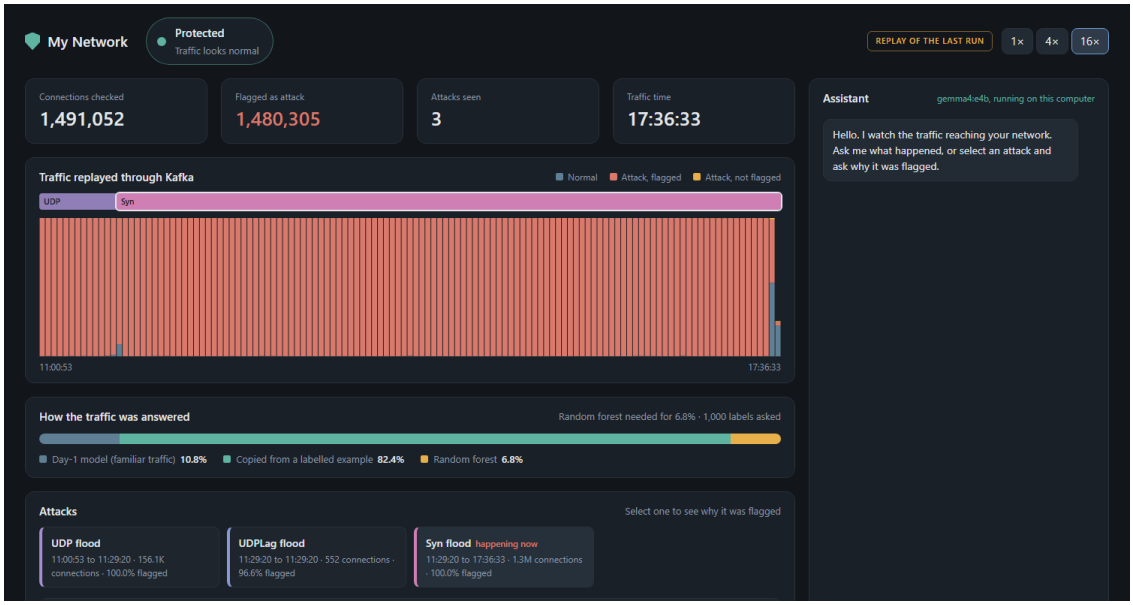


Fig. 8. The "My Network" website: status, connections checked and flagged, the replayed traffic by time, how the traffic was answered, and the attacks seen.

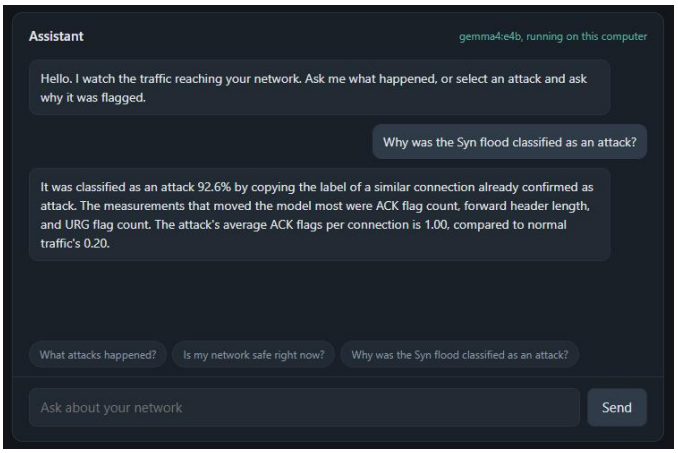


Fig. 9. The assistant (Gemma, running locally) explaining why the Syn flood was flagged, using the routing shares and feature reasons from the live job.

[4] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, 2022, pp. 3971–3988.

[5] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop distributed file system," in *Proc. IEEE 26th Symp. Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10.

[6] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. 9th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2012, pp. 15–28.

[7] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi, and M. Zaharia, "Spark SQL: Relational data processing in Spark," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2015, pp. 1383–1394.

[8] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar, "MLlib: Machine learning in Apache Spark," *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.

[9] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, Athens, Greece, 2011.

[10] M. Armbrust, T. Das, J. Torres, B. Yavuz, S. Zhu, R. Xin, A. Ghodsi, I. Stoica, and M. Zaharia, "Structured streaming: A declarative API for real-time applications in Apache Spark," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2018, pp. 601–613.

[11] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2017, pp. 253–262.

[12] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019.

[13] M. J. Awan, U. Farooq, H. M. A. Babar, A. Yasin, H. Nobanee, M. Hussain, O. Hakeem, and A. M. Zain, "Real-time DDoS attack detection system using big data approach," *Sustainability*, vol. 13, no. 19, p. 10743, 2021.

[14] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[15] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[16] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[17] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[18] B. Bahmani, B. Moseley, A. Vattani, R. Kumar, and S. Vassilvitskii, "Scalable k-means++," *Proceedings of the VLDB Endowment*, vol. 5, no. 7, pp. 622–633, 2012.

[19] T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[20] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2001, pp. 511–518.

[21] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[22] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification

across space and time," in *Proc. 28th USENIX Security Symp.*, 2019, pp. 729–746.

[23] B. Settles, "Active learning literature survey," University of Wisconsin–Madison, Computer Sciences Technical Report 1648, 2009.

[24] R. Jordaney, K. Sharad, S. K. Dash, Z. Wang, D. Papini, I. Nouretdinov, and L. Cavallaro, "Transcend: Detecting concept drift in malware classification models," in *Proc. 26th USENIX Security Symp.*, 2017, pp. 625–642.

[25] L. Yang, W. Guo, Q. Hao, A. Ciptadi, A. Ahmadzadeh, X. Xing, and G. Wang, "CADE: Detecting and explaining concept drift samples for security applications," in *Proc. 30th USENIX Security Symp.*, 2021, pp. 2327–2344.

[26] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.